MPEC Reference and Known-Truth Validation Tutorial

Mathematical Programming with Equilibrium Constraints

econirl package documentation

May 2026

RTD page: https://econirl.readthedocs.io/en/latest/estimators/mpec.html **Source:** https://github.com/rawatpranjal/EconIRL/tree/main

1 Purpose

MPEC estimates the same structural dynamic discrete choice likelihood as NFXP, but it changes the numerical problem. NFXP eliminates the value function by solving the Bellman fixed point inside every likelihood evaluation. MPEC keeps the value function as an optimizer variable and imposes the Bellman fixed point as an equality constraint.

This document is a known-truth validation tutorial. It does not use real data. The synthetic DGP supplies the true reward, transition law, value function, policy, and Type A/B/C counterfactual oracle objects. The validation question is whether the MPEC implementation recovers these known objects on a medium-scale finite panel while satisfying the Bellman constraints.

2 Source Papers

The primary source is Su and Judd (2012), which formulates structural dynamic estimation as constrained optimization. Their core contribution is to solve for (θ, V) jointly:

$$\max_{\theta, V} \ell(\theta, V) \quad \text{subject to} \quad V = T_\theta V.$$

The constraint is the equilibrium condition. At any feasible point, MPEC and NFXP evaluate the same likelihood.

Iskhakov, Rust, Schjerning, and Seo (2016) give the caution. MPEC is elegant and easy to specify, but the constrained problem has dimension $|\theta| + |\mathcal{S}|$ and can become less robust than specialized NFXP-NK as the fixed-point dimension grows or β approaches one. In this package, MPEC is treated as a structural estimator for manageable tabular state spaces, not as a universal large-state solver.

3 When to Use MPEC

Use MPEC when the state space is tabular, transitions are known or first-stage estimated, the Bellman constraint dimension is moderate, and you want a direct constrained-optimization formulation of the structural likelihood. It is also useful for auditing NFXP because both estimators target the same MLE but reach it through different numerical geometry.

Do not use MPEC as the main recovery tool for large state spaces, nearly singular Bellman Jacobians, or high-discount problems unless the constraint solver has been stress-tested. The relevant failure mode is not an inner Bellman loop failing; it is an optimizer reporting success while constraint violation, stationarity, or standard errors are not credible. The known-truth gates below check those objects directly.

4 Model

The observed data are state-action-next-state trajectories

$$\{(s_{it}, a_{it}, s_{i,t+1}) : i = 1, \ldots, N,\ t = 1, \ldots, T\}.$$

Flow payoff is linear in known features:

$$u_\theta(s, a) = \phi(s, a)^\top \theta.$$

Transitions are

$$P_a(s, s') = \Pr(s_{t+1} = s' \mid s_t = s, a_t = a),$$

the discount factor is β, and the type-I extreme-value shock scale is σ. Given V, the choice-specific value is

$$Q_\theta(s, a; V) = u_\theta(s, a) + \beta \sum_{s'} P_a(s, s') V(s').$$

The logit policy implied by (θ, V) is

$$\pi_{\theta,V}(a \mid s) = \frac{\exp(Q_\theta(s, a; V)/\sigma)}{\sum_b \exp(Q_\theta(s, b; V)/\sigma)}.$$

The soft Bellman operator is

$$T_\theta V(s) = \sigma \log \sum_a \exp\left(\frac{u_\theta(s, a) + \beta \sum_{s'} P_a(s, s') V(s')}{\sigma}\right).$$

5 MPEC Estimator

NFXP solves

$$\hat{\theta}_{\text{NFXP}} = \arg\max_\theta \sum_{i,t} \log \pi_\theta(a_{it} \mid s_{it}), \quad \text{where } V_\theta = T_\theta V_\theta.$$

MPEC solves the constrained problem

$$(\hat{\theta}, \hat{V}) = \arg\max_{\theta, V} \sum_{i,t} \log \pi_{\theta,V}(a_{it} \mid s_{it}) \quad \text{subject to} \quad c(\theta, V) = V - T_\theta V = 0.$$

The implementation uses scipy SLSQP with JAX-supplied objective gradients and constraint Jacobians. It initializes V at the Bellman fixed point of the initial θ, which gives SLSQP a feasible starting point. Each SQP step works with the joint variable $x = (\theta, V)$ and the stacked equality constraint $c(x) \in \mathbb{R}^{|\mathcal{S}|}$. There is no nested Bellman solve inside the likelihood objective.

At convergence, standard errors use the same implicit score as NFXP. The Bellman derivative is

$$(I - \beta P_\pi) \frac{\partial V}{\partial \theta} = \sum_a \pi(a \mid s) \phi(s, a),$$

where $P_\pi(s, s') = \sum_a \pi(a \mid s) P_a(s, s')$. The package uses the outer product of per-observation scores for robust standard errors in the known-truth validation run.

6 Identification Checks

MPEC and NFXP share the same structural identification requirements, with one additional numerical requirement from constrained optimization.

1. **Reward rank.** The action-dependent feature matrix must have full column rank after dropping zero rows.

2. **Action support.** Each action must appear in the finite panel so the likelihood can discipline payoff differences.
3. **State coverage.** The canonical tabular validation observes every state in the regular state space.
4. **Normalization.** The reward scale is fixed at $\sigma = 1$, and the synthetic DGP uses an exit action and absorbing state as a zero-reward anchor.
5. **Constraint qualification.** The Bellman constraint Jacobian must be numerically well behaved at the solution. The generated run reports the final Bellman residual and gates on it.

7 Known-Truth DGP

The canonical MPEC validation cell is `canonical_low_action`. It is the same low-dimensional action-dependent structural benchmark used for NFXP and CCP.

Table 1: Canonical MPEC DGP.

Quantity	Value
Regular states	20
Absorbing states	1
Total states	21
Actions	3
Exit action	2
Discount factor	0.95
Shock scale	1.0
Reward mode	action-dependent
Reward dimension	low
State mode	low-dimensional
Simulated individuals	2,000
Periods per individual	80
Observations	160,000

The action-dependent features are

$$\phi(s,a) = \begin{cases} (1, x_s, 0, 0) & a = 0, \\ (0, 0, 1, x_s) & a = 1, \\ (0, 0, 0, 0) & a = 2. \end{cases}$$

The exit action has zero reward and sends the process to the absorbing state. The true parameter vector is:

Table 2: True reward parameters.

Parameter	Truth
`action_0_intercept`	0.10
`action_0_progress`	0.50
`action_1_intercept`	0.00
`action_1_progress`	-0.20

8 Pre-Estimation Diagnostics

Run diagnostics before fitting. For MPEC, the estimator is structural, so the rank and normalization checks are not optional.

```
from experiments.known_truth import (
    SimulationConfig,
```

```
    build_known_truth_dgp,
    get_cell,
    run_pre_estimation_diagnostics,
    simulate_known_truth_panel,
)

cell = get_cell("canonical_low_action")
dgp = build_known_truth_dgp(cell.dgp_config)
panel = simulate_known_truth_panel(
    dgp,
    SimulationConfig(n_individuals=2000, n_periods=80, seed=42),
)
diagnostics = run_pre_estimation_diagnostics(dgp, panel)

assert diagnostics.passed
assert diagnostics.feature_rank == diagnostics.num_features
assert diagnostics.is_action_dependent
assert diagnostics.state_action_coverage == 1.0
```

9 Run the Tutorial

From the repository root:

```
PYTHONPATH=src:. python -m experiments.known_truth \
    --estimator MPEC \
    --cell-id canonical_low_action \
    --output-dir outputs/known_truth \
    --show-progress \
    --verbose
```

The command builds the DGP, simulates the panel with `tqdm` progress, runs pre-estimation diagnostics, fits MPEC with finite robust standard errors, evaluates reward/value/Q/policy recovery, solves Type A/B/C counterfactual oracles, writes one `result.json`, and raises if any non-smoke hard gate fails.

Direct estimator usage:

```
from experiments.known_truth import (
    build_known_truth_dgp,
    get_cell,
    simulate_known_truth_panel,
)
from econirl.estimation.mpec import MPECConfig, MPECEstimator

cell = get_cell("canonical_low_action")
dgp = build_known_truth_dgp(cell.dgp_config)
panel = simulate_known_truth_panel(dgp, cell.simulation_config)

estimator = MPECEstimator(
    config=MPECConfig(
        solver="sqp",
        outer_max_iter=200,
        tol=1e-8,
        constraint_tol=1e-6,
    ),
    se_method="robust",
    compute_hessian=True,
    verbose=True,
)
```

```
summary = estimator.estimate(
    panel=panel,
    utility=dgp.utility(),
    problem=dgp.problem,
    transitions=dgp.transitions,
)
```

10 Generated Validation Results

This section is generated by `mpec_run.py` from the `experiments.known_truth` harness and the canonical known-truth DGP.

10.1 Pre-Estimation Checks

Table 3: Pre-estimation checks from the known-truth MPEC run.

Check	Value	Status
Feature rank	4 / 4	pass
Feature condition number	4.512	pass
Transition row error	2.42×10^{-8}	pass
Observed states	21 / 21	pass
State-action coverage	1.000	pass
Action shares	0.345, 0.330, 0.325	pass
Minimum action share	0.325	pass
Exit/absorbing anchor	true	pass

10.2 Estimator Fit

The medium-scale validation run uses 2,000 individuals, 80 periods per individual, and 160,000 observations.

Table 4: Known-truth MPEC run summary.

Quantity	Value
Converged	true
SQP iterations	19
Log-likelihood	-174875.7719
Estimation time	3.39 seconds
Solver	slsqp
SciPy success	true
Final Bellman constraint violation	7.72×10^{-12}
Standard errors finite	true

Table 5: Parameter recovery.

Parameter	Truth	Estimate	SE	Error
`action_0_intercept`	0.100000	0.083894	0.029336	-0.016106
`action_0_progress`	0.500000	0.528522	0.035890	0.028522
`action_1_intercept`	0.000000	-0.014461	0.036733	-0.014461
`action_1_progress`	-0.200000	-0.200511	0.052502	-0.000511

Table 6: Recovery metrics.

Metric	Value
Parameter RMSE	0.017905
Parameter relative RMSE	0.065378
Parameter cosine similarity	0.998867
Reward RMSE	0.009694
Value RMSE	0.019445
Q RMSE	0.022437
Policy KL	9.21×10^{-5}
Policy total variation	0.005697
Policy max state L1	0.018905

Table 7: Hard recovery gates.

Gate	Threshold	Value	Status
converged	true	true	pass
constraint_violation	≤ 0.000001	0.000000	pass
standard_errors_finite	true	true	pass
parameter_cosine	≥ 0.980000	0.998867	pass
parameter_relative_rmse	≤ 0.150000	0.065378	pass
policy_tv	≤ 0.030000	0.005697	pass
value_rmse	≤ 0.100000	0.019445	pass
q_rmse	≤ 0.100000	0.022437	pass
type_a_regret	≤ 0.050000	0.000213	pass
type_b_regret	≤ 0.050000	0.000362	pass
type_c_regret	≤ 0.050000	0.000086	pass

10.3 Counterfactual Recovery

Table 8: Counterfactual recovery against known oracle objects.

Counterfactual	Policy TV	Policy KL	Value RMSE	Regret
Type A	0.005109	7.56×10^{-5}	0.000238	0.000213
Type B	0.005457	8.20×10^{-5}	0.000363	0.000362
Type C	0.003549	3.56×10^{-5}	0.000114	0.000086

11 Debugging Gaps

If MPEC fails to recover truth, debug in this order.

1. **Constraint violation.** Check `metadata["final_constraint_violation"]`. A good likelihood value is not enough if $V \neq T_\theta V$.
2. **Solver success.** Check both `summary.converged` and the underlying SLSQP success flag.
3. **Feature rank.** If rank is below the number of reward parameters, θ is not identified.
4. **Action support.** If one action is rare or missing, the likelihood has weak information about payoff differences.
5. **Standard errors.** Non-finite standard errors mean the score outer product or implied information matrix is singular. Do not publish that run as a full structural validation.

6. **Initialization.** MPEC is sensitive to feasible starts. The implementation initializes V at the Bellman fixed point of the initial θ.
7. **Scale and anchor.** If the logit scale or exit-action anchor is changed, reward levels can drift.
8. **Soft gates.** Do not accept a non-smoke run that only warns. Known-truth validation should pass hard gates or raise.

12 Practical Defaults

Recommended non-smoke validation settings:

```
MPECEstimator(
    config=MPECConfig(
        solver="sqp",
        outer_max_iter=200,
        tol=1e-8,
        constraint_tol=1e-6,
    ),
    se_method="robust",
    compute_hessian=True,
)
```

Use smoke settings only for code-path tests. A smoke test shows that the estimator runs and returns finite objects; it is not evidence of recovery.

13 Implementation Paths

- Estimator: `src/econirl/estimation/mpec.py`
- Known-truth harness: `experiments/known_truth.py`
- Fast validation tests: `tests/test_known_truth.py`
- Master plan: `papers/econirl_package_jss/plans/README.md`
- MPEC alignment audit: `papers/econirl_package_jss/plans/alignment/04_mpec.md`

14 References

Su, Che-Lin, and Kenneth L. Judd (2012). Constrained Optimization Approaches to Estimation of Structural Models. *Econometrica*, 80(5), 2213–2230.

Iskhakov, Fedor, John Rust, Bertel Schjerning, and Kyoungwon Seo (2016). Comment on "Constrained Optimization Approaches to Estimation of Structural Models." *Econometrica*, 84(1), 365–370.